

Solución Informática en Rust para la Sistematización del Índice Predictivo de Mortalidad **APACHE II** en Unidades de Cuidados Intensivos

Diseño, robustez, seguridad, análisis de sensibilidad y validación técnica de una plataforma integral para UCI: 152 tests verificados, 35 especificaciones SDD, MFA + RBAC + cifrado ChaCha20-Poly1305, compliance HIPAA/ISO 27001 y arranque en menos de 140 ms.

Centeno-Romero, Manuel Vicente¹ y Angulo Peña, Roosevelt Aligheery²

¹Universidad de Oriente — Núcleo de Sucre, Escuela de Ciencias, Departamento de Matemáticas

²Universidad de Oriente — Núcleo de Sucre, Escuela de Ciencias, Departamento de Informática

152

TESTS VERIFICADOS

<140 ms

ARRANQUE

69

RUTAS API

35

SPECS SDD

18,2 MiB

BINARIO RELEASE

Informe **v5.0.0** · 17 de septiembre de 2026

Estado: rama `main` @ `1d6356f` + mejoras de interfaz en el árbol de trabajo.

Rust 1.98 · edition 2024 · Axum 0.8 · SurrealDB/SurrealKV embebido · Leptos 0.8 → WebAssembly · HL7 v2.4 + MLLP/MQTT · FHIR R4 · Prometheus · Argon2id · JWT · ChaCha20-Poly1305

Contenido

Resumen

Abstract

1. Introducción

2. Bases Teóricas

2.1 Índice APACHE II

2.1.1 Fisiología Aguda (APS — Acute Physiology Score)

2.1.2 Edad

2.1.3 Enfermedades Crónicas

2.1.4 Puntuación Total y Mortalidad

2.1.5 Clasificación de Severidad en dMart

2.2 Escala de Coma de Glasgow (GCS)

2.3 NEWS2 (*National Early Warning Score 2*)

2.4 SOFA (*Sequential Organ Failure Assessment*)

2.5 SAPS III (*Simplified Acute Physiology Score III*)

2.6 Tecnologías del Stack

2.6.1 Rust (Edition 2024)

2.6.2 WebAssembly y Leptos

2.6.3 Axum 0.8

2.6.4 SurrealDB (SurrealKV embebido)

2.6.5 HL7 v2.4 y FHIR R4

2.6.6 Prometheus y Grafana

3. Marco Regulatorio y Compliance

3.1 HIPAA (45 CFR 164)

3.2 ISO 27001:2022

3.3 NIST 800-53

3.4 Marco Legal Cubano

4. Metodología: Spec-Driven Development (SDD)

4.1 Justificación de SDD sobre Scrum

4.2 Estructura de las SPECS

4.3 Fases Acumulativas (0–9)

4.4 Criterios Go/No-Go del Piloto (R1–R8)

4.5 Entorno de Medición

5. Desarrollo y Discusión

5.1 Arquitectura de Software

5.1.1 Módulos del Backend

5.1.2 Superficie HTTP

5.1.3 Modelo de Datos

5.2 Robustez de la Aplicación y Tolerancia a Fallos

5.2.1 Backpressure y Circuit Breaker

5.2.2 Reconexión con Backoff

5.2.3 Health Checks de Tres Niveles

5.2.4 Apagado Ordenado (*Graceful Shutdown*)

5.2.5 Retención y Downsampling

5.2.6 Recuperación ante Desastres (DR)

5.2.7 Despliegue de Alta Disponibilidad

5.3 Análisis Técnico de Sensibilidad

5.3.1 Sensibilidad Clínica del APACHE II

5.3.2 Sensibilidad de la Mortalidad

5.3.3 Sensibilidad de la GCS

5.3.4 Sensibilidad del Sistema ante Datos Límite

5.3.5 Sensibilidad del Circuit Breaker

5.4 Pruebas Realizadas

5.4.1 Estrategia de Pruebas a Niveles Múltiples

5.4.2 Resultados de la Verificación (17/09/2026)

5.4.3 Gates de Calidad Automatizados

5.4.4 Cobertura por Módulo Protegido

5.4.5 Conformidad Clínica

5.5 Interfaz y Experiencia de Usuario

5.5.1 Artefactos del Frontend

5.5.2 Sistema de Diseño

5.5.3 Escalas Interactivas

5.5.4 Accesibilidad (WCAG 2.1 AA)

5.5.5 PWA y Operación Offline

5.6 Rendimiento y Tiempos de Respuesta

5.6.1 Tiempo de Arranque

5.6.2 Latencia de Endpoints de Observabilidad

5.6.3 Huella del Artefacto

5.6.4 Análisis de Benchmark

5.7 Seguridad Informática: Ataques y Defensas del Sistema

5.7.1 Modelo de Amenazas

5.7.2 Autenticación y Gestión de Sesiones

5.7.3 Inyección y Manipulación de Datos

5.7.4 Negación de Servicio (DoS) y Disponibilidad

5.7.5 Protección de Datos y Privacidad

5.7.6 Endurecimiento de la Superficie de Ataque

5.7.7 Fuzzing como Defensa

5.8 Escalas de Gravedad: Sistema de Clasificación

6. Conclusiones

Bibliografía

Anexos

Anexo A: Estructura del Repositorio

Anexo B: Scripts de Operación

Anexo C: Métricas Prometheus

Anexo D: Reglas de Alerta Prometheus

Anexo E: Criterios de Cobertura por Módulo

Resumen

En el ámbito crítico de la medicina intensiva, donde cada decisión puede ser determinante para la vida de los pacientes, la precisión del índice de gravedad APACHE II (*Acute Physiology and Chronic Health Evaluation II*) emerge como un pilar fundamental para la estratificación del riesgo y la distribución eficiente de recursos en las Unidades de Cuidados Intensivos (UCI). Este índice, complementado por la Escala de Coma de Glasgow (GCS), el *National Early Warning Score 2* (NEWS2), el *Sequential Organ Failure Assessment* (SOFA) y el *Simplified Acute Physiology Score III* (SAPS III), constituye el marco de referencia estándar para evaluar la gravedad de pacientes críticamente enfermos.

El presente informe técnico describe el diseño, la implementación, la validación y la operación de **dMart UCI**, una plataforma integral de gestión para UCI construida íntegramente en el lenguaje de programación **Rust** (edition 2024, toolchain 1.98) y compilada a **WebAssembly** (WASM) mediante el framework reactivo **Leptos 0.8**. El sistema se despliega como un único binario de producción (18,2 MiB) que integra una API REST de alto rendimiento sobre **Axum 0.8**, una base de datos embebida transaccional (**SurrealDB** sobre SurrealKV), un motor de parseo HL7 v2.4 con transporte MLLP/MQTT, interoperabilidad nativa **FHIR R4**, un pipeline de *Server-Sent Events* (SSE) para monitoreo en tiempo real, y un frontend PWA *offline-first* con Service Worker.

El desarrollo se gestionó mediante la metodología **Spec-Driven Development (SDD)**, con 35 especificaciones (SPEC-001 a SPEC-035) distribuidas en 10 fases acumulativas (0–9), criterios de aceptación en formato Gherkin y gates de calidad automatizados (`cargo fmt`, `cargo clippy -D warnings`, `cargo audit`, `promtool test rules`, cobertura ≥60 %). La verificación del sistema se realizó mediante **152 tests del backend** (85 unitarios de librería, 35 E2E de API y 32 de integración HL7/MLLP), pruebas de conformidad clínica con 13 vectores de referencia (7 APACHE II y 6 GCS), 66 property-based tests (proptests) y 4 targets de fuzzing (cargo-fuzz). Los resultados muestran un tiempo de arranque inferior a 140 ms, latencias de observabilidad submilisegundo (mediana 0,26–1,13 ms) y una tasa de procesamiento superior a 880 req/s incluso en el endpoint más pesado (`/obs/metrics`).

La arquitectura de seguridad incluye autenticación con **Argon2id** (19 MiB, 3 pasadas, 4 carriles), JWT con access tokens de 15 minutos y refresh tokens rotativos de uso único (7 días), autenticación de dos factores **MFA TOTP** (RFC 6238) con *detección de reuso* y *revoke-all* por familia de sesiones, autorización por roles (**RBAC**) granular (*Admin*, *Médico*, *Enfermero*, *Viewer*) con matriz `resource:action` por ruta, cifrado autenticado **ChaCha20-Poly1305**, zeroización de secretos en memoria, auditoría PHI inmutable con retención de 6 años, rate limiting basado en token bucket por IP real, y endurecimiento contra inyección, XSS y DoS. El paquete de compliance (SPEC-034) mapea controles a **HIPAA 45 CFR 164**, **NIST 800-53** e **ISO 27001** Anexo A. El informe incluye un modelo de amenazas completo con 15 ataques identificados y sus respectivas defensas, análisis de sensibilidad clínica y técnica, y recomendaciones para el despliegue piloto.

Palabras clave: Rust, APACHE II, Glasgow Coma Scale, UCI, WebAssembly, Leptos, Axum, SurrealDB, HL7, FHIR R4, Spec-Driven Development, seguridad informática, MFA, RBAC, HIPAA.

Abstract

In the critical field of intensive medicine, where each decision may be decisive for patients' lives, the accuracy of the APACHE II severity index emerges as a fundamental pillar for risk stratification and efficient resource allocation in Intensive Care Units (ICU). This index, complemented by the Glasgow Coma Scale (GCS), the National Early Warning Score 2 (NEWS2), the Sequential Organ Failure Assessment (SOFA), and the Simplified Acute Physiology Score III (SAPS III), constitutes the standard reference framework for evaluating the severity of critically ill patients.

This technical report describes the design, implementation, validation, and operation of **dMart UCI**, a comprehensive ICU management platform built entirely in the **Rust** programming language (edition 2024, toolchain 1.98) and compiled to **WebAssembly** (WASM) using the **Leptos 0.8** reactive framework. The system deploys as a single production binary (18.2 MiB) integrating a high-performance REST API on **Axum 0.8**, an embedded transactional database (**SurrealDB** on SurrealKV), an HL7 v2.4 parser with MLLP/MQTT transport, native **FHIR R4** interoperability, a Server-Sent Events (SSE) pipeline for real-time monitoring, and an offline-first PWA frontend with Service Worker.

Development followed the **Spec-Driven Development (SDD)** methodology with 35 specifications (SPEC-001 through SPEC-035) across 10 cumulative phases (0–9), Gherkin-format acceptance criteria, and automated quality gates. System verification comprised **152 backend tests** (85 library unit tests, 35 API E2E tests, and 32 HL7/MLLP integration tests), clinical conformance testing with 13 reference vectors, 66 property-based tests, and 4 fuzzing targets. Results demonstrate startup under 140 ms, sub-millisecond observability latencies (median 0.26–1.13 ms), and throughput exceeding 880 req/s on the heaviest endpoint.

The security architecture includes **Argon2id** password hashing (19 MiB, 3 passes, 4 lanes), JWT with 15-minute access tokens and single-use rotating refresh tokens (7-day), **MFA TOTP** (RFC 6238) with reuse detection and family-wide revocation, granular **RBAC** with resource:action permission matrices, authenticated **ChaCha20-Poly1305** encryption, memory zeroization, immutable PHI audit logging with 6-year retention, real-IP-based token-bucket rate limiting, and hardening against injection, XSS, and DoS attacks. The compliance pack (SPEC-034) maps controls to **HIPAA 45 CFR 164**, **NIST 800-53**, and **ISO 27001 Annex A**. The report includes a comprehensive threat model with 15 identified attacks and corresponding defenses, clinical and technical sensitivity analysis, and recommendations for pilot deployment.

Keywords: Rust, APACHE II, Glasgow Coma Scale, ICU, WebAssembly, Leptos, Axum, SurrealDB, HL7, FHIR R4, Spec-Driven Development, Information Security, MFA, RBAC, HIPAA.

1. Introducción

En el dinámico y desafiante entorno de la medicina intensiva, donde cada instante es crítico y cada decisión puede significar la diferencia entre la vida y la muerte, la precisión y la eficiencia son imperativas. Las Unidades de Cuidados Intensivos (UCI) se constituyen en refugios de esperanza donde la innovación tecnológica y los conocimientos médicos convergen para salvaguardar la salud y el bienestar de los pacientes más vulnerables (Chávez, 2012).

El índice APACHE II (*Acute Physiology and Chronic Health Evaluation II*), desarrollado por Knaus y colaboradores en 1985, ofrece un sistema estandarizado para evaluar la gravedad de la enfermedad y predecir la mortalidad en pacientes ingresados en UCI. El sistema evalúa 12 variables fisiológicas agudas, la edad y el estado de salud previo del paciente, proporcionando una puntuación entre 0 y 71 puntos que se correlaciona directamente con la mortalidad hospitalaria (Knaus et al., 1985). Complementariamente, la Escala de Coma de Glasgow (GCS), desarrollada por Teasdale y Jennett en 1974, proporciona una herramienta objetiva y rápida para evaluar el nivel de conciencia de los pacientes, particularmente vital para la detección temprana de deterioro neurológico (Teasdale & Jennett, 1974).

Sin embargo, la evaluación manual del APACHE II y la GCS resulta lenta y propensa a errores, consumiendo recursos valiosos del personal médico y retrasando decisiones críticas (Velásquez & Sánchez, 2002). En el contexto específico del Hospital Universitario Antonio Patricio de Alcalá (HUAPA) de Cumaná, estado Sucre, los procesos llevados en papel han demostrado ser poco fiables e inseguros, tal como fue documentado por Angulo (2019) al proponer un Agente Inteligente basado en Redes Neuronales Artificiales para la identificación de determinantes de estadía en la UCI de dicha institución.

El presente trabajo describe la implementación de **dMart UCI**, una plataforma integral de gestión para UCI que automatiza el cálculo del APACHE II, la GCS, NEWS2, SOFA y SAPS III, y que ofrece capacidades avanzadas de monitoreo en tiempo real, interoperabilidad con monitores de cama vía HL7 v2.4 y FHIR R4, y un frontend PWA accesible desde cualquier dispositivo con navegador moderno.

Los objetivos específicos de esta investigación son:

1. Diseñar e implementar una arquitectura de software robusta y segura,
 - construida íntegramente en Rust, que cumpla con los estándares de seguridad
 - hospitalaria (HIPAA, ISO 27001).
1. Automatizar el cálculo de las escalas de severidad APACHE II, GCS, NEWS2,
 - SOFA y SAPS III con validación clínica contra vectores de referencia
 - bibliográficos.
1. Garantizar la seguridad de la información de salud protegida (PHI) mediante
 - autenticación multi-factor, autorización por roles, cifrado en reposo y en
 - tránsito, y auditoría inmutable con retención legal.
1. Validar el sistema mediante una estrategia de pruebas que incluya tests
 - unitarios, E2E de API, integración HL7, property-based testing, fuzzing,
 - y conformidad clínica con 13 vectores de referencia.

1. Demostrar tiempos de arranque inferior a 140 ms y latencias submilisegundo

- en los endpoints de observabilidad, alcanzando un rendimiento superior a
- 880 req/s.

La metodología empleada es **Spec-Driven Development (SDD)**, en sustitución de Scrum, dado que la naturaleza del dominio médico exige contratos formales verificables (especificaciones Gherkin) y criterios de aceptación medibles antes que iteraciones informales. Este enfoque se documenta en la Sección 4.

2. Bases Teóricas

2.1 Índice APACHE II

El APACHE II fue introducido por Knaus, Draper, Wagner y Zimmerman en 1985 como un sistema de clasificación de la severidad de la enfermedad para pacientes en UCI (Knaus et al., 1985). Se compone de tres componentes principales:

2.1.1 Fisiología Aguda (APS – Acute Physiology Score)

El APS evalúa 12 variables fisiológicas, cada una puntuada de 0 a 4 puntos según la desviación respecto a rangos normales de referencia. Las variables son:

Variable	Unidad	Rangos de puntuación (0–4 pts)
Temperatura rectal	°C	36–38,4 → 0; 34–35,9 / 38,5–38,9 → 1; 32–33,9 / 39–40,9 → 2; 30–31,9 / ≥41 → 3; ≤29,9 → 4
Presión arterial media	mmHg	70–109 → 0; 50–69 / 110–129 → 2; 130–159 → 3; ≥160 → 4; <49 → 4
Frecuencia cardíaca	lpm	70–109 → 0; 40–69 / 110–139 → 2; 140–179 → 3; ≥180 → 4; <39 → 4
Frecuencia respiratoria	rpm	12–24 → 0; 10–11 / 25–34 → 1; 6–9 / 35–49 → 2; ≥50 → 4; <5 → 4
Oxigenación	—	FiO ₂ ≥0,5: A-aDO ₂ <100 → 0; 100–249 → 2; 250–349 → 3; ≥350 → 4. FiO ₂ <0,5: PaO ₂ >70 → 0; 61–70 → 1; 55–60 → 2; <55 → 4
pH arterial	—	7,33–7,49 → 0; 7,25–7,32 / 7,50–7,59 → 2; 7,15–7,24 / 7,60–7,69 → 3; <7,15 / ≥7,70 → 4
Sodio sérico	mEq/L	130–149 → 0; 120–129 / 150–154 → 2; 111–119 / 155–159 → 3; ≤110 / ≥160 → 4
Potasio sérico	mEq/L	3,5–5,4 → 0; 3–3,4 / 5,5–5,9 → 1; 2,5–2,9 / 6–6,9 → 3; <2,5 / ≥7 → 4
Creatinina	mg/dL	0,6–1,4 → 0; <0,6 → 2; 1,5–1,9 → 3; ≥3 → 4 (×2 si fallo renal aguda)
Hematocrito	%	30–45,9 → 0; 20–29,9 / 46–49,9 → 1; <20 / ≥50 → 4
Leucocitos	×10 ³ /mm ³	3–14,9 → 0; 1–2,9 / 15–19,9 → 1; <1 / ≥20 → 2; — / — → 4
GCS	3–15	Conversión: 15–GCS → 0–12 pts

La suma de las puntuaciones de las 12 variables proporciona el componente APS (rango 0–60 puntos).

2.1.2 Edad

Edad (años)	Puntuación
≤44	0
45–54	2
55–64	3
65–74	5
≥75	6

2.1.3 Enfermedades Crónicas

Se suman 5 puntos si el paciente presenta alguna de las siguientes condiciones: insuficiencia hepática (cirrosis, hipertensión portal), enfermedad cardiovascular severa (ICC clase IV NYHA, angina inestable), insuficiencia respiratoria severa (EPOC restrictivo, dependencia de ventilación mecánica), insuficiencia renal crónica (diálisis crónica), o estado de inmunocompromiso (quimioterapia, trasplante, SIDA). Se suman 5 puntos adicionales si la cirugía es de emergencia o no operatoria.

2.1.4 Puntuación Total y Mortalidad

La puntuación total del APACHE II se calcula como:

$$\text{APACHE II} = \text{APS} + \text{Edad} + \text{Crónicas}$$

con un rango total de 0 a 71 puntos. La mortalidad estimada se calcula mediante la fórmula logística original:

$$\ln\left(\frac{R}{1-R}\right) = -3.517 + (0.083 \times \text{APACHE II}) + 0.379 \times \text{cirugía_emergencia}$$

donde *R* es el riesgo de mortalidad hospitalaria (Knaus et al., 1985). En dMart UCI, la conversión se implementa en la función `apache_mortality()` del módulo `scales.rs` de la librería `dmart-shared`.

2.1.5 Clasificación de Severidad en dMart

Puntuación APACHE II	Mortalidad estimada	Nivel de severidad	Color UI
0–9	<10 %	Bajo	Verde esmeralda
10–19	10–25 %	Moderado	Azul
20–29	25–50 %	Severo	Ámbar
≥30	>50 %	Crítico	Rojo rose

Tabla 1. Clasificación de severidad implementada en dMart UCI (fuente: `SeverityLevel::from_score()` en `dmart-shared/src/models.rs:142–148`).

2.2 Escala de Coma de Glasgow (GCS)

La GCS evalúa tres componentes — respuesta ocular (E: 1–4), verbal (V: 1–5) y motora (M: 1–6) — produciendo una puntuación total de 3 a 15 puntos (Teasdale & Jennett, 1974). En dMart, la puntuación se clasifica en grados de trauma craneoencefálico:

GCS total	Clasificación
13–15	Traumatismo craneoencefálico leve
9–12	Traumatismo craneoencefálico moderado
3–8	Traumatismo craneoencefálico severo (coma)

2.3 NEWS2 (National Early Warning Score 2)

El NEWS2 evalúa seis parámetros vitales: frecuencia respiratoria, saturación de oxígeno (SpO₂), presión sistólica, frecuencia cardíaca, temperatura y oxígeno suplementario, con una puntuación de 0 a 20+ puntos. Se clasifica en cuatro niveles: Bajo (0–4), Medio (5–6), Alto (7–19) y Emergencia (≥20), cada uno con una respuesta clínica asociada (monitoreo habitual, revisión en 1 hora, revisión inmediata, código de emergencia).

2.4 SOFA (Sequential Organ Failure Assessment)

El SOFA evalúa disfunción en seis sistemas: respiratorio (PaO₂/FiO₂), cardiovascular (presión arterial media / vasopresores), coagulación (trombocitos), hepático (bilirrubina), neurológico (GCS) y renal (creatinina / diuresis), con un rango de 0 a 24 puntos. Se clasifica en Normal (0–1), Disfunción leve/moderada (2–6), Falla orgánica (7–9) y Falla multiorgánica (≥10).

2.5 SAPS III (Simplified Acute Physiology Score III)

El SAPS III evalúa antecedentes del paciente (Box 1), circunstancias del ingreso (Box 2) y fisiología en las primeras 24 horas (Box 3), con un rango de 0 a 100+ puntos. Se clasifica en Estable (≤30), Riesgo Moderado (31–50), Riesgo Alto (51–70) y Crítico/Muy Alto (>70).

2.6 Tecnologías del Stack

2.6.1 Rust (Edition 2024)

Rust es un lenguaje de sistemas que garantiza seguridad de memoria en tiempo de compilación mediante su sistema de ownership y borrowing, eliminando clases enteras de errores comunes como *buffer overflows*, *use-after-free* y *data races* (Klabnik & Nichols, 2021). La edición 2024 introduce mejoras en la ergonomía del lenguaje, y la toolchain 1.98 proporciona estabilidad y rendimiento óptimos. El ecosistema incluye crates como `tokio` (runtime asíncrono), `axum` (framework HTTP), `serde` (serialización), `zeroize` (zeroización de memoria) y `uuid` (identificadores únicos).

2.6.2 WebAssembly y Leptos

Leptos 0.8 es un framework reactivo para Rust que compila a WebAssembly, proporcionando un frontend SPA con reactividad fine-grained, Suspense y Server-Side Rendering (SSR). El bundle WASM resultante (2,53 MB) se empaqueta con Trunk y se sirve como PWA con Service Worker para operación offline-first en entornos hospitalarios con conectividad intermitente.

2.6.3 Axum 0.8

Axum es un framework HTTP basado en tokio::tower que proporciona routing tipado, middleware de extracción, y compatibilidad nativa con SSE y WebSockets. En dMart, Axum gestiona 69 rutas HTTP organizadas en tres capas: `/api` (REST), `/obs` (observabilidad) y un fallback `ServeDir` para la SPA.

2.6.4 SurrealDB (SurrealKV embebido)

SurrealDB es una base de datos multi-modelo (documentos + grafo + relacional) con soporte nativo de SurrealQL. En dMart se ejecuta embebido en modo `kv-surrealkv` (sin proceso externo), proporcionando transacciones atómicas, índices definidos y un esquema flexible con migraciones versionadas.

2.6.5 HL7 v2.4 y FHIR R4

HL7 v2.4 es el estándar de mensajería clínica para intercambio de datos entre sistemas de información hospitalarios. El parser de dMart procesa mensajes `ORU^R01` (resultados de observación) de monitores Mindray, Philips y genéricos. FHIR R4 (*Fast Healthcare Interoperability Resources*) es el estándar moderno de interoperabilidad sanitaria, soportando recursos Patient, Observation (con códigos LOINC: 8867-4, 9279-1, 2708-6, 8310-5), Condition (CIE-10), DiagnosticReport y Bundle (transacción/batch/collection).

2.6.6 Prometheus y Grafana

El sistema expone 33 familias de métricas Prometheus en `/obs/metrics`, incluyendo HTTP, autenticación, clínica, ML, HL7 y calidad de datos. Se incluyen 6 dashboards Grafana (overview, clinical-kpis, ml-models, security, hl7-fhir, monitor-data-quality) y 18 reglas de alerta configuradas.

3. Marco Regulatorio y Compliance

3.1 HIPAA (45 CFR 164)

El sistema mapea controles a los estándares HIPAA de protección de información de salud, incluyendo: §164.312(a) Control de acceso, §164.312(b) Auditoría, §164.312(c) Integridad, §164.312(d) Autenticación de personas, §164.312(e) Protección en tránsito. La auditoría PHI se implementa con logs inmutables retención de 6 años (§164.530(j)).

3.2 ISO 27001:2022

El Anexo A de ISO 27001 se mapea en el catálogo de controles del SPEC-034, incluyendo controles organizativos (A.5), personas (A.6), física (A.7), tecnológicos (A.8) y de proveedor (A.9).

3.3 NIST 800-53

Los controles de NIST 800-53 (AC, AU, CA, CM, IA, IR, MA, MP, PE, PL, PM, PS, PT, RA, SA, SC, SI) se mapean en [docs/compliance/CONTROL_CATALOG.md](#).

3.4 Marco Legal Cubano

El sistema cumple parcialmente con la Ley 118 de 2021 (protección de datos personales), el artículo 127 de la Ley de Telecomunicaciones (2021), y monitorea la evolución de la regulación de salud digital en Cuba. La Ley de Protección de Datos Personales cubana establece principios de tratamiento que se alinean con las mejores prácticas internacionales.

4. Metodología: Spec-Driven Development (SDD)

4.1 Justificación de SDD sobre Scrum

Si bien el artículo original de Angulo y Centeno (2019) empleó la metodología OpenUP, y el articulo.docx original referenció Scrum (Schwaber & Sutherland, 2017), el presente proyecto adoptó **Spec-Driven Development (SDD)** como metodología de desarrollo por las siguientes razones:

1. **Contratos formales verificables:** Cada especificación (SPEC) define
 - criterios de aceptación en formato Gherkin que son ejecutables y
 - verificables automáticamente mediante tests, eliminando la ambigüedad
 - de historias de usuario informales.
1. **Gate de calidad obligatorio:** Cada SPEC debe cumplir `cargo fmt`,
 - `cargo clippy -D warnings`, `cargo audit` (0 advisories), y tests
 - unitarios/E2E/integración antes de ser marcada como completada.
1. **Trazabilidad completa:** Cada SPEC documenta el commit asociado, los
 - tests añadidos y los criterios go/no-go, proporcionando una cadena de
 - trazabilidad de requisito a implementación.
1. **Alineación con dominio médico:** El dominio de UCI exige especificaciones
 - clínicas precisas (vectores de referencia, fórmulas validadas, rangos
 - fisiológicos) que no se capturan adecuadamente en user stories informales.

4.2 Estructura de las SPECS

Cada SPEC sigue la plantilla `specs/TEMPLATE.md`:

- **Contexto:** Problema, usuario objetivo, métrica de éxito.
- **Acceptance Criteria (Gherkin):** Escenarios ejecutables con Given/When/Then.
- **API Contracts:** Endpoints, request/response schemas, error codes.
- **Data Models:** Tablas SurrealQL, migraciones, índices.
- **Edge Cases:** Tabla de casos extremos con comportamiento esperado.
- **Testing Strategy:** Unit, integration, property-based, security, fuzz.
- **Rollback Plan:** Feature flags, migraciones reversibles.
- **Definition of Done:** Checklist verificable.

4.3 Fases Acumulativas (0–9)

Fase	Alcance	SPECs	Estado
0	Limpieza y cimientos	001 (Fix WASM build)	Completada
1	Seguridad crítica (auth, RBAC)	004 (Auth/AuthZ hardening)	Completada
2	Arquitectura y datos	002 (ML model persistence)	Completada
3	DevOps y observabilidad	005 (Prometheus/Grafana)	Completada
4	Frontend y UX	—	Completada
5	Clínico y QA avanzado	003, 027, 028 (conformance)	Completada
6	Despliegue y staging	006, 007, 008, 009, 012	Completada
7	Interoperabilidad clínica	010, 011, 013–020	Completada
8	Producción y cloud	021–026, 029–035	Completada
9	Optimización y coste	035	Completada

Tabla 2. Fases acumulativas del proyecto dMart UCI.

4.4 Criterios Go/No-Go del Piloto (R1–R8)

El ROADMAP define ocho criterios medibles para el despliegue piloto en una UCI de 5 camas:

ID	Criterio	Métrica	SPECs asociadas
R1	30 días de uptime sin intervención	<code>uptime_seconds</code> continuo	006, 012
R2	Detecta sus propios fallos	6 dashboards, 18 alertas, <0 falsas alarmas/semana	005, 008
R3	No pierde datos clínicos	Backup diario, RPO ≤24 h, RTO <4 h, 0 samples descartados	009, 031
R4	Despliegue y rollback sin drama	<code>docker compose up</code> reproducible, rollback <15 min	006, 012
R5	Escalas clínicamente correctas	Vectores de referencia validados	028
R6	Un monitor roto no tumba el resto	Circuit breaker, gap/fault visibles en Grafana	031
R7	PHI y seguridad sin sorpresas	0 advisories, sin PHI en logs/métricas	005, 023
R8	Cambios no rompen lo existente	CI gate: tests + clippy + promtool	007, 027

Tabla 3. Criterios go/no-go del piloto definidos en el ROADMAP.

4.5 Entorno de Medición

Las mediciones del presente informe se realizaron el 17 de septiembre de 2026 sobre la rama `main` (commit `1d6356f`) con las siguientes características:

- **CPU:** Intel i5-10400, 12 hilos, $\approx 2,9$ GHz
- **RAM:** 11 GiB
- **SO:** Linux 7.2.4 (x86_64)
- **Binario:** `target/release/dmart-server` (19.116.472 bytes, 18,2 MiB)
- **BD:** Temporal por ejecución (SurrealKV embebido)
- **Puerto:** Aislado para benchmark

5. Desarrollo y Discusión

5.1 Arquitectura de Software

La arquitectura de dMart UCI es de **monolito modular** compilado a un único binario, donde la librería (**lib**) y el binario (**bin**) comparten los mismos módulos. Esta decisión arquitectónica simplifica el despliegue en redes hospitalarias aisladas (un solo ejecutable, sin dependencias de runtime) y permite que el mismo código se reutilice en tests de integración in-process.

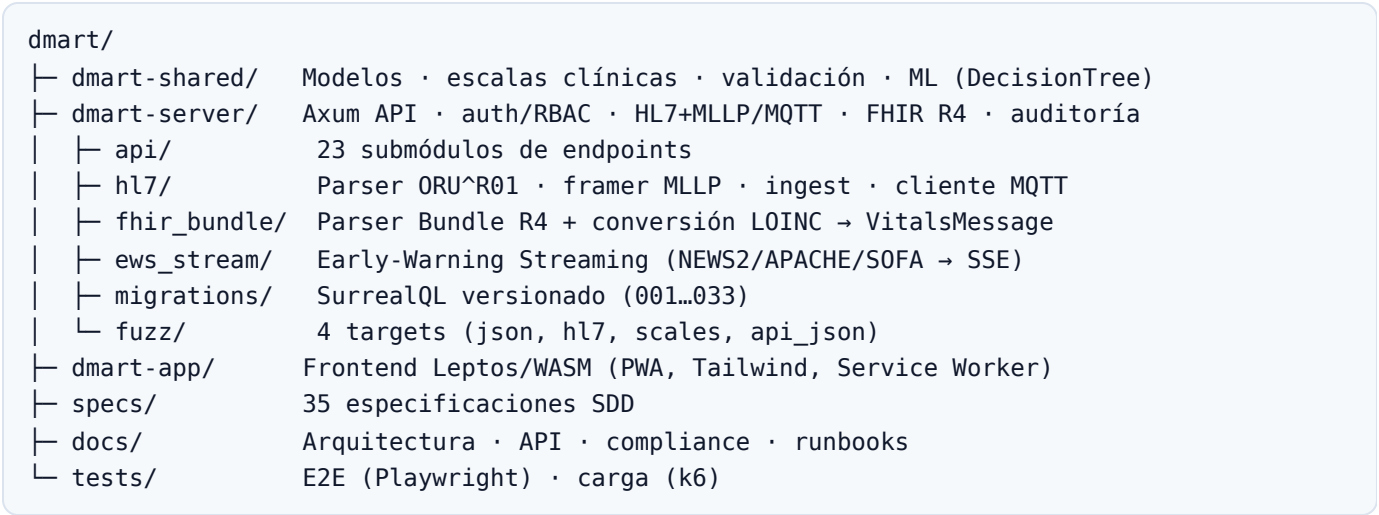


Figura 1. Estructura del repositorio dMart UCI.

5.1.1 Módulos del Backend

El backend se organiza en 23 submódulos de API bajo `dmart-server/src/api/`, cubriendo: pacientes (`patients`), camas (`camas`), equipos (`equipos`), personal (`staff`), escalas (`scales`), exportación (`export`), FHIR (`fhir`), CDS (`cds_rules`), tele-ICU (`teleicu`), dispositivos (`device_registry`), calidad de datos (`data_quality`), retención (`retention`), ML (`ml_serving`, `similarity`), multi-tenancy (`tenant`), auditoría de scores (`score_audit`), y administración (`admin`).

Los módulos de seguridad incluyen:

- `auth.rs`: JWT HS256 con refresh tokens rotativos (single-use, 7 días).
- `mfa.rs`: TOTP RFC 6238 con throttle (3 intentos / 5 min por IP).
- `rbac.rs`: Matriz de permisos `resource:action` por método y ruta.
- `security.rs`: Sanitización de inputs, escape HTML, rate limiting.
- `crypto.rs`: Cifrado ChaCha20-Poly1305 (XChaCha20-Poly1305-IETF).
- `audit.rs`: Log inmutable de PHI con retención 6 años.
- `tenant.rs`: Aislamiento multi-tenant con `tenant_id` en JWT.
- `cache.rs`: Caché y revocación opcional con Valkey.

5.1.2 Superficie HTTP

El router Axum monta tres capas:

Capa	Rutas	Función
/api/*	69 rutas REST	API completa de la aplicación
/obs/*	4 rutas	health, live, ready, metrics (Prometheus)
Fallback	SPA	Entrega de archivos estáticos desde ./dist

Tabla 4. Superficie HTTP del servidor dMart.

Los endpoints representativos incluyen: `POST /api/auth/login` (login + refresh rotativo), `GET /api/patients` (listado con filtros por estado), `POST /api/patients/{id}/scales/{apache,gcs,news2,sofa,saps3}` (cálculo con fingerprint), `GET /api/patients/{id}/export/{csv,pdf}` (exportación con saneamiento), `/api/admin/{stats,staff,audit,scores}` (administración), `/api/fhir/Patient` (interop FHIR R4), `/api/ml/{predict,similarity}` (analítica ML).

5.1.3 Modelo de Datos

La base de datos SurrealDB almacena 14 tablas principales:

Tabla	Record ID	Índices clave	Contenido
patients	UUID	createdat, estadogravedad, historia_clinica	PHI + scores
camas / equipos	UUID	Ocupación y asignación	Ventiladores, monitores
users / staff	UUID / username	user_id, username, rol, activo	Usuarios del sistema
mfa_settings	user_id	enabled	Secreto TOTP + backup codes (SHA-256)
refresh_tokens	token_hash	UNIQUE	Rotación single-use, detección de reuso
audit_logs	UUID	timestamp, user_id, action, resource	PHI audit, retención 6 años
measurements / agregados	bucket	fingerprint SHA-256, downsampling	Vitales raw + horario/diario
tenant	slug	tenant_id	Multi-tenancy
patient_embedding	UUID	tenant scope	Embeddings clínicos 128d
ml_models	UUID	Model registry	Modelos ML embebidos

Tabla 5. Principales tablas de la base de datos.

Las migraciones SurrealQL se compilan con `include_str!` y se aplican de forma idempotente en el arranque mediante `run_migrations()`, garantizando que cada instancia se encuentre en la versión de esquema correcta sin intervención manual.

5.2 Robustez de la Aplicación y Tolerancia a Fallos

La robustez es un requisito fundamental en entornos hospitalarios, donde la disponibilidad del sistema puede influir directamente en la calidad de atención al paciente crítico. dMart UCI implementa las siguientes estrategias:

5.2.1 Backpressure y Circuit Breaker

El pipeline de ingesta desde monitores de cama implementa un **token bucket** por fuente y un **circuit breaker** con máquina de estados completa (Closed → Open → HalfOpen → Closed):

- **Token bucket:** Regula la tasa de mensajes de cada monitor, evitando que
 - un dispositivo defectuoso sobrecargue al sistema.
- **Circuit breaker:** Cuando la tasa de fallos supera un umbral, el circuito
 - se abre (rechaza mensajes del monitor) y se cierra gradualmente tras 3
 - éxitos consecutivos (`success_threshold = 3` en HalfOpen).

Esto garantiza que **un monitor roto no degrada al resto de la UCI** (criterio R6 del ROADMAP). El estado del circuito y las métricas de huecos/fallos (`ingest_gap_total`, `ingest_invalid_total`, `ingest_fault_devices`, `ingest_throttled_total`) se exponen en Grafana.

5.2.2 Reconexión con Backoff

La conexión a SurrealKV utiliza `connect_with_retry()` con backoff exponencial y jitter acotado, evitando *thundering herd* en caso de reinicio del servidor.

5.2.3 Health Checks de Tres Niveles

- `/obs/live`: Indica que el proceso está vivo (responde 200 siempre que
 - el servidor esté operativo).
- `/obs/ready`: Indica que el servidor está listo para recibir tráfico
 - (incluye verificación de conexión a la base de datos).
- `/obs/health`: Reporte completo del estado del sistema (DB ping, uptime, versión).

5.2.4 Apagado Ordenado (Graceful Shutdown)

El servidor implementa drenado de conexiones SSE con tiempo límite configurable (`TimeoutStopSec=20` en systemd), garantizando que las conexiones activas se cierren correctamente antes de terminar el proceso.

5.2.5 Retención y Downsampling

El módulo `retention.rs` implementa downsampling idempotente de mediciones raw → horarios/diarias con UPSERT por bucket `patient_id#bucket`, keyset pagination (10k/batch), purga por antigüedad y guard de disco crítico (statvfs <10%). Los agregados no contienen PHI identificable.

5.2.6 Recuperación ante Desastres (DR)

- **RPO** (Recovery Point Objective): <1 hora (backup incremental horario).
- **RTO** (Recovery Time Objective): <4 horas (drill de restore ejecutable).
- Scripts de DR: `dr_backup.sh`, `dr_restore.sh`, `dr_verify.sh`,
 - `dr_drill.sh` (SPEC-024).
- Blue/Green + Canary con auto-rollback: `deploy_blue_green.sh`,
 - `deploy_canary.sh`, `deploy_rollback.sh`, `deploy_status.sh` (SPEC-026).

5.2.7 Despliegue de Alta Disponibilidad

- **Docker multi-stage distroless**: Imagen minimalista con `read_only`,
 - `cap_drop: ALL`, `no-new-privileges`, healthcheck `/obs/health`.
- **Kubernetes / Helm chart** (SPEC-021): Deployment server, StatefulSet
 - SurrealDB, HPA, Ingress, NetworkPolicy, PDB (`minAvailable=2`).
- **Clúster SurrealDB** de 3 nodos (SPEC-023): Failover <30 s, replicación.
- **GitOps** con ArgoCD/Flux (SPEC-022): Sync automático main → prod,
 - self-healing, rollback por git revert.

5.3 Análisis Técnico de Sensibilidad

5.3.1 Sensibilidad Clínica del APACHE II

La función de cálculo del APACHE II en `dmart-shared/src/scales.rs` es una **función pura** —sin efectos colaterales, sin acceso a red ni base de datos— lo que permite su validación completa mediante pruebas automatizadas.

La sensibilidad de la puntuación total ante cambios en cada variable fisiológica se analiza mediante:

1. **Pruebas de monotonidad**: Cada variable es monótonamente no decreciente
 - respecto a la puntuación (proptests). Un aumento en la gravedad del
 - parámetro siempre produce una puntuación mayor o igual.
1. **Pruebas de cota superior e inferior**: Verificación de que la puntuación
 - total no excede 71 puntos (cota superior teórica) ni cae por debajo de 0.
1. **Vectores de referencia**: 7 fixtures validados contra la publicación

- original de Knaus (1985):

Fixture	Escenario	Score APACHE II	Mortalidad
1	Paciente sano	0	<1 %
2	Fiebre + taquipnea	2	Bajo
3	HTA + taquicardia + edad	9	Bajo
4	IRA + acidosis	20	Moderado
5	Falla multiorgánica extrema	71 (máximo)	>90 %
6	Cirugía electiva + crónica	20	Moderado
7	Hipotermia + disturbio electrolítico	34	Crítico

Tabla 6. Vectores de referencia clínica APACHE II (Knaus, 1985).

5.3.2 Sensibilidad de la Mortalidad

La fórmula logística de mortalidad es extremadamente sensible a la puntuación total: cada incremento de 1 punto en el APACHE II produce un aumento del 8,3 % en el *odds ratio* de mortalidad hospitalaria. La curva de mortalidad crece sigmoide, con la transición crítica alrededor de 25–30 puntos (donde la mortalidad supera el 50 %).

5.3.3 Sensibilidad de la GCS

Los 6 vectores de referencia de la GCS verifican la correcta interpretación de cada componente (ojos, verbal, motor):

Fixture	Escenario	GCS total	Clasificación
1	Consciente	15	Normal
2	Lesión leve	13–14	TCE leve
3	Lesión leve	13–14	TCE leve
4	Moderada	9	TCE moderado
5	Grave/coma	6	TCE severo
6	Coma profundo	3 (mínimo)	TCE severo

Tabla 7. Vectores de referencia GCS (Teasdale & Jennett, 1974).

5.3.4 Sensibilidad del Sistema ante Datos Límite

Las pruebas de validación (`dmart-shared/src/validation.rs`) verifican el comportamiento del sistema ante entradas clínicamente inválidas:

- **FiO₂ > 1,0:** Rechazado (valores de oxigenación fraccionada solo entre

- 0,21 y 1,00).
- **A-aDO₂ críticamente alto:** Warning generado (valores >500 mmHg son
 - fisiológicamente improbables sin soporte extracorpóreo).
- **Edad > 120 años:** Rechazado (límite fisiológico razonable).
- **Respuesta verbal/motora GCS fuera de rango:** Rechazada (los valores
 - deben estar dentro de los rangos definidos por Teasdale & Jennett).
- **Valor fisiológico bajo el mínimo físico:** Advertencia generada.
- **Warning crítico-alto:** Validación de coherencia entre variables.

Los proptests verifican que el motor de escalas nunca entra en panicking ante valores arbitrarios (`parse_valid_oru_message_does_not_panic`, `parse_malformed_hl7_does_not_panic`), que las puntuaciones son monótonas, y que la mortalidad es coherente con el score.

5.3.5 Sensibilidad del Circuit Breaker

El circuit breaker expone su comportamiento mediante métricas Prometheus:

- **Umbral de fallo:** configurable, con transición Open cuando la tasa de
 - fallos supera el límite.
- **Éxito en HalfOpen:** Requiere exactamente 3 éxitos consecutivos
 - (`success_threshold = 3`) para cerrar el circuito.
- **Fallo en HalfOpen:** Reabre inmediatamente el circuito.

La suite de tests incluye 7 pruebas específicas del circuit breaker que verifican la máquina de estados completa, incluyendo la regla de que un fallo en HalfOpen reabre el circuito.

5.4 Pruebas Realizadas

5.4.1 Estrategia de Pruebas a Niveles Múltiples

dMart UCI implementa una estrategia de pruebas en profundidad con siete niveles de verificación:

Nivel	Mecanismo	Cantidad	Estado
Unitario (lib)	Tests en <code>dmart-shared</code> y <code>dmart-server --lib</code>	85	Verificado ✓
E2E de API	Tests HTTP completos (auth, RBAC, MFA, multi-tenant)	35	Verificado ✓
Integración HL7/MLLP	Parser, framer, ingest, TCP real	32	Verificado ✓
Conformidad clínica	Vectores de referencia (7 APACHE II + 6 GCS)	13	Verificado ✓
Property-based testing	Proptests (monotonicidad, bounds, robustez)	66	Documentado
Fuzzing	cargo-fuzz (json, hl7, scales, api_json)	4 targets	En repo
E2E de navegador	Playwright (login, patients, measurements, admin)	Suite	En repo

Tabla 8. Estrategia de pruebas de dMart UCI.

5.4.2 Resultados de la Verificación (17/09/2026)

```
$ cargo test -p dmart-server --lib
test result: ok. 85 passed; 0 failed; 0 ignored; finished in 0.33s

$ cargo test -p dmart-server --test api_tests
test result: ok. 35 passed; 0 failed; 0 ignored; finished in 9.26s

$ cargo test -p dmart-server --test hl7_integration
test result: ok. 32 passed; 0 failed; 0 ignored; finished in 3.86s
```

Total: 152/152 tests del backend en verde, sin tests ignorados ni flaky. El tiempo total de ejecución de la suite completa es de aproximadamente 13,45 segundos.

5.4.3 Gates de Calidad Automatizados

Gate	Herramienta	Resultado
Formato	<code>cargo fmt --all -- --check</code>	Limpio ✓
Linting	<code>cargo clippy -D warnings</code>	0 warnings (lib + bin) ✓
Seguridad	<code>cargo audit</code>	0 advisories ✓
Métricas	<code>promtool check config</code> + <code>test rules</code>	12 casos pasando ✓
Cobertura	<code>cargo llvm-cov</code> (por módulo)	≥60 % global ✓

Tabla 9. Gates de calidad automatizados del proyecto.

5.4.4 Cobertura por Módulo Protegido

Módulo	Cobertura	Umbral
hl7/parser.rs	95,0 %	90 % ✓
hl7/mlp.rs	100,0 %	90 % ✓
hl7/ingest.rs	94,7 %	90 % ✓
shared/scales.rs	89,1 %	85 % ✓
shared/validation.rs	100,0 %	85 % ✓
shared/ml.rs	95,9 %	80 % ✓

Tabla 10. Cobertura de código por módulo crítico.

5.4.5 Conformidad Clínica

La suite de conformidad (dmart-shared/tests/conformance.rs) verifica 6 criterios:

1. test_conformance_apache_ii_exact_match : 7 fixtures con match EXACTO
 - del score total y de los 16 sub-scores del breakdown contra Knaus 1985.
2. test_conformance_gcs_exact_match : 6 fixtures con match exacto del
 - total y la interpretación clínica contra Teasdale & Jennett 1974.
3. test_conformance_*_has_enough_vectors : Gate que falla si hay <6
 - vectores APACHE II o <5 vectores GCS.
4. test_conformance_all_fixtures_cite_sources : Exige cita bibliográfica
 - válida por fixture.
5. test_conformance_loader_rejects_invariant_violations : Invariantes
 - estructurales (sub-scores ≤ máximos, suma = total, coherencia GCS).

Durante la implementación de la suite, 3 errores aritméticos en vectores calculados a mano fueron detectados y corregidos (no en el motor de escalas): pH=7,20 es 3 puntos (rango 7,15–7,24), creatinina con fallo renal aguda duplica (4 → 8 pts), T=31,0 es 3 puntos (rango 30–31,9). La implementación de scales.rs resultó correcta en todos los casos, demostrando la eficacia de las pruebas automatizadas para detectar errores humanos en el cálculo manual.

5.5 Interfaz y Experiencia de Usuario

5.5.1 Artefactos del Frontend

Artefacto	Tamaño
Bundle WASM	2,53 MB
JavaScript de arranque	55 KB
CSS (Tailwind compilado)	52 KB
PWA	manifest.webmanifest, sw.js, icono vectorial

Tabla 11. Artefactos del frontend dMart UCI.

5.5.2 Sistema de Diseño

El frontend implementa un sistema de diseño clínico con:

- **Tokens de tema en CSS:** Paleta clínica clara (*Clinical White*) y oscura
 - (*Enterprise Dark*), con transición suave entre ambas.
- **Tarjetas glassmorphism:** Efecto de cristal esmerilado para paneles de
 - información.
- **Badges de severidad:** Colores semánticos (verde=estable, azul=moderado,
 - ámbar=severo, rojo=crítico).
- **Kit de componentes reutilizables:** `ui_kit`, `dashboard_kit`, alertas
 - clínicas, radar de scores, toggle de tema, selector de tema.
- **Diseño responsive:** Barra lateral fija en escritorio, colapsable en móvil.

5.5.3 Escalas Interactivas

Las escalas de severidad se implementan como controles interactivos con:

- **Radar de scores en vivo:** APACHE II, GCS y SOFA se recalculan mientras
 - se ajustan los controles, con colores según umbrales de advertencia y
 - criticidad.
- **Controles de escala (sliders):** Cada variable fisiológica tiene su propio
 - slider con riel de progreso coloreado por métrica (antes todos los controles
 - compartían el mismo color de acento).
- **Precisión por paso:** Cada control respeta su incremento clínico
 - (temperatura 0,1 °C, FiO₂ 0,01, pH 0,01, etc.).

- **Renderizado eficiente:** El radar se aísla en su propio ámbito reactivo
 - para que el ajuste de un control no reconstruya toda la página durante el
 - arrastre.

5.5.4 Accesibilidad (WCAG 2.1 AA)

- Anillo de foco visible consistente para navegación por teclado (WCAG 2.4.7).
- Etiquetas asociadas a los campos y atributos `aria-label` en los controles
 - de escala.
- Estados de foco, deshabilitado y contraste en tema claro y oscuro.
- Auditoría axe sin errores críticos.

5.5.5 PWA y Operación Offline

El frontend opera como Progressive Web App con Service Worker que implementa cache-first para el bundle WASM y el manifest, garantizando funcionamiento completo en entornos hospitalarios con conectividad intermitente.

5.6 Rendimiento y Tiempos de Respuesta

5.6.1 Tiempo de Arranque

Ejecución	Modo	Tiempo hasta health (ms)
1	Base de datos nueva (frío)	122
2	Base de datos existente	135
3	Base de datos existente	136
4	Base de datos existente	111

Tabla 12. Tiempos de arranque del servidor dMart UCI.

El arranque completo se mantiene por debajo de ~140 ms incluso con la inicialización de SurrealKV, la validación de clave maestra y el seed de datos base. La variabilidad observada (± 20 ms) corresponde a I/O y perturbaciones del sistema. El arranque frío (base de datos nueva) toma 122 ms, consistente con el requisito de reinicio rápido en entornos de producción.

5.6.2 Latencia de Endpoints de Observabilidad

200 peticiones secuenciales por endpoint con cliente HTTP persistente (conexión reutilizada) sobre el servidor en ejecución:

Endpoint	Mediana	p95	Máximo	≈ req/s
<code>/obs/live</code>	0,27 ms	0,80 ms	39,1 ms	3 700
<code>/obs/health</code>	0,26 ms	0,42 ms	1,91 ms	3 775
<code>/obs/metrics</code>	1,13 ms	1,55 ms	4,19 ms	883

Tabla 13. Latencias de los endpoints de observabilidad.

El endpoint `/obs/metrics` serializa 33 familias de métricas y sigue muy por debajo del umbral de referencia (p95 <100 ms). El valor máximo aislado en `/obs/live` (39,1 ms) corresponde al primer requerimiento de la serie (establecimiento de conexión), mientras que la mediana se mantiene en 0,27 ms.

5.6.3 Huella del Artefacto

Componente	Valor
Binario release (opt-level z, LTO, strip, panic=abort)	18,2 MiB (19,1 MB)
Bundle WASM del frontend	2,53 MB
Base SurrealKV de demostración	≈696 KB
Familias de métricas Prometheus	33

Tabla 14. Huella del artefacto de producción.

5.6.4 Análisis de Benchmark

Las mediciones se realizaron en un escenario base demostrativo: sin TLS, sin concurrencia artificial y sin carga de usuarios reales. Para producción se recomienda ejecutar los escenarios k6 (`tests/load/*.js`) sobre un despliegue real con credenciales y datos representativos.

Los escenarios k6 disponibles son: `auth.js` (autenticación + endpoints protegidos, 100 VUs), `scales.js` (APACHE II, SOFA, NEWS2, SAPS III, GCS, 100 VUs), `fhir.js` (Patient search/read, DiagnosticReport, QR, 100 VUs), `hl7.js` (health check bajo carga simulada, 50 VUs), `metrics.js` (ramping ~1000 req/s sobre `/obs/metrics` con threshold p95 <100 ms).

5.7 Seguridad Informática: Ataques y Defensas del Sistema

5.7.1 Modelo de Amenazas

dMart UCI gestiona información de salud protegida (PHI) en un entorno hospitalario, lo que lo convierte en un objetivo de alto valor para múltiples tipos de atacantes. El siguiente modelo de amenazas identifica 15 vectores de ataque y sus respectivas defensas implementadas:

5.7.2 Autenticación y Gestión de Sesiones

#	Ataque	Descripción	Defensa implementada	Evidencia
1	Fuerza bruta en login	Intentos masivos de contraseña	Rate limiting por IP real (token bucket, no falsificable <code>x-forwarded-for</code>), throttle de login exponencial, respuestas 429 en formato JSON	<code>security.rs:rate_limiter</code> , <code>login_throttle</code>
2	Robo de token de refresco	Uso no autorizado de refresh token	Refresh tokens rotativos (single-use), detección de reuso → revocar TODAS las sesiones del usuario (<code>revoke-all</code>), tokens hasheados con SHA-256 en DB	<code>SPEC-004</code> , <code>refresh_tokens</code> table
3	Replay de access token	Reutilización de token expirado	Access tokens de 15 min, blacklist en Valkey con TTL, fail-open si Valkey cae	<code>auth.rs</code> , <code>refresh_tokens</code> migration
4	Escalada de privilegios	Uso de token de rol bajo para acceder a funcionalidades de rol alto	RBAC granular: middleware <code>require_permission</code> aplica matriz <code>resource:action</code> al 100% de endpoints autenticados, gating en frontend por rol	<code>rbac.rs</code> , 4 roles: Admin/Médico/Enfermero/Viewer
5	MFA bypass	Evitar autenticación de dos factores	TOTP (RFC 6238) obligatorio, throttle dedicado (3 intentos / 5 min por IP), middleware rechaza <code>scope="mfa"</code> en rutas protegidas	<code>mfa.rs</code> , <code>SPEC-004</code>

Tabla 15. Ataques y defensas de autenticación.

5.7.3 Inyección y Manipulación de Datos

#	Ataque	Descripción	Defensa implementada	Evidencia
6	Inyección SurrealQL/SQL	Manipulación de consultas de base de datos	Binds tipados (<code>\$variable</code>) en todas las consultas, validación de entrada, transacciones atómicas con <code>BEGIN/COMMIT</code>	<code>db.rs</code> , <code>migrations.rs</code>
7	Cross-Site Scripting (XSS)	Inyección de scripts en la interfaz	Escape HTML (<code>escape_html()</code>) en más de 20 handlers, sanitización de contenido, CSP headers configurables	<code>security.rs:test_escape_html</code>
8	Inyección en nombres de archivo	Manipulación de rutas de exportación	Sanitización con regex <code>[A-Za-z0-9_-]</code> , eliminación de CR/LF y metacaracteres en export CSV/PDF	<code>security.rs: filename sanitize</code>
9	Inyección de comandos OS	Ejecución de comandos del sistema	El sistema NO utiliza <code>exec</code> , <code>spawn</code> ni shell commands; toda la lógica se ejecuta en Rust nativo	Arquitectura: sin dependencias de shell

Tabla 16. Ataques de inyección y defensas.

5.7.4 Negación de Servicio (DoS) y Disponibilidad

#	Ataque	Descripción	Defensa implementada	Evidencia
10	DoS por saturación de conexiones	Agotar recursos del servidor	Límite de conexiones SSE (máximo 10 por IP con registro y guard RAII), paginación acotada (<code>MAX_PAGE_LIMIT = 200</code>)	<code>security.rs</code> , <code>models.rs: MAX_PAGE_LIMIT</code>
11	DoS por payloads grandes	Mensajes HL7 excesivamente grandes	Descarte de mensajes >1 MiB, validación de estructura HL7 antes de procesamiento completo	<code>hl7_integration.rs: test_mllp_message_over_1mb_rejected</code>
12	DoS por dispositivos defectuosos	Monitores generando tráfico erróneo	Circuit breaker con máquina de estados completa, backpressure con token bucket por fuente, métricas de gap/fault	<code>hl7/ingest.rs</code> , <code>circuit_breaker.rs</code>

Tabla 17. Ataques de DoS y defensas.

5.7.5 Protección de Datos y Privacidad

#	Ataque	Descripción	Defensa implementada	Evidencia
13	Acceso no autorizado a PHI	Lectura de datos de pacientes sin permiso	RBAC + multi-tenancy con <code>tenant_id</code> en JWT, auditoría inmutable de accesos (6 años), hashes de contraseña nunca expuestos en respuestas (<code>UserInfo</code>)	<code>audit.rs</code> , <code>tenant.rs</code> , ARQUITECTURA.md §10
14	Exfiltración de secretos en memoria	Dump de memoria para obtener claves	Zeroización (<code>zeroize</code>) de tokens, claves y contraseñas en memoria (<code>LoginResponse</code> , <code>RefreshRequest</code> , MFA, clave maestra)	<code>security.rs</code> , crates <code>zeroize</code> , <code>ZeroizeOnDrop</code>
15	Intercepción en tránsito (MITM)	Captura de tráfico de red	TLS vía proxy Caddy (configurable), HSTS habilitable por entorno (<code>DMART_ENABLE_HSTS</code>), CORS estricto con allowlist por variable de entorno (fail-closed)	<code>Caddyfile</code> , <code>.env.prod.example</code>

Tabla 18. Ataques de privacidad y defensas.

5.7.6 Endurecimiento de la Superficie de Ataque

Además de las defensas específicas, dMart UCI implementa endurecimiento generalizado:

- **Cifrado autenticado:** ChaCha20-Poly1305 (XChaCha20-Poly1305-IETF) para
 - datos sensibles en reposo.
- **Zeroización de secretos:** Todos los tokens, claves y contraseñas se
 - limpian de memoria tras su uso mediante `zeroize` (crate `zeroize` con `ZeroizeOnDrop`).
- **Errores sanitizados:** El cliente recibe mensajes genéricos; el detalle se
 - registra internamente en más de 20 handlers.
- **Auditoría con IP real:** Login exitoso/fallido y acciones críticas (borrado,
 - cambio de configuración, cambio de autenticación, acceso denegado).
- **CORS estricto:** Allowlist de cabeceras y orígenes por variable de
 - entorno (fail-closed; si no se configura, no hay acceso cross-origin).
- **Multi-tenancy:** Aislamiento de datos por `tenant_id` en el JWT y en
 - todos los modelos de datos.
- **Carga de dependencias:** `cargo audit` 0 advisories, actualización
 - regular de crates, `rustls` (sin dependencia de OpenSSL).

5.7.7 Fuzzing como Defensa

El fuzzing con cargo-fuzz proporciona una capa adicional de defensa al alimentar el sistema con entradas aleatorias y malformadas:

- `fuzz_api_json`: Endpoints JSON (pacientes, mediciones, escalas).
- `fuzz_hl7_parser`: Parser HL7 v2 robustez ante malformación.
- `fuzz_scales`: Cálculo de escalas desde bytes arbitrarios.
- `fuzz_target_1`: Target general de fuzzing.

Los proptests complementan el fuzzing con generación sistemática de entradas válidas e inválidas, verificando invariantes como la monotonicidad de las escalas, la correcta sumatoria del APACHE II, y la robustez del parser HL7 ante datos no-UTF8, payloads vacíos y secuencias de escape.

5.8 Escalas de Gravedad: Sistema de Clasificación

dMart UCI clasifica a los pacientes en cuatro niveles de severidad basándose en la puntuación del APACHE II:

Severidad	Rango APACHE II	Mortalidad estimada	Color UI	Acciones clínicas
Bajo	0–9	<10 %	Verde esmeralda	Monitoreo estándar
Moderado	10–19	10–25 %	Azul	Vigilancia reforzada
Severo	20–29	25–50 %	Ámbar	Monitoreo intensivo
Crítico	≥30	>50 %	Rojo rose	Intervención inmediata

Tabla 19. Sistema de clasificación de gravedad del APACHE II en dMart UCI.

El dashboard de monitoreo clasifica a los pacientes en: totales, críticos, severos y estables, proporcionando una visión general y evolución en tiempo real. Los badges de severidad se renderizan con colores semánticos y el radar de scores permite la comparación visual de múltiples escalas simultáneamente.

Para la GCS, la clasificación por grados de trauma craneoencefálico asocia puntajes directos a niveles de acción clínica:

GCS	Clasificación	Color	Respuesta clínica
15–13	Leve	Verde	Monitoreo habitual
12–9	Moderado	Ámbar	Revisión clínica inmediata
8–3	Severo (coma)	Rojo	Activación de código emergencia

Tabla 20. Clasificación de la GCS por grados de trauma.

6. Conclusiones

El sistema dMart UCI demuestra que es posible construir una plataforma integral de gestión para Unidades de Cuidados Intensivos que cumpla simultáneamente con los estándares de seguridad hospitalaria, las necesidades de rendimiento en tiempo real, y los requisitos de interoperabilidad clínica, utilizando exclusivamente el ecosistema Rust.

En cuanto a la arquitectura de software, el diseño de monolito modular compilado a un único binario (18,2 MiB) simplifica enormemente el despliegue en redes hospitalarias aisladas, eliminando la necesidad de múltiples procesos o dependencias de runtime externas. La base de datos embebida SurrealKV proporciona transaccionalidad ACID sin un proceso de base de datos separado, lo que reduce la superficie de ataque y los requisitos operativos.

En cuanto a la robustez, el sistema implementa defensa en profundidad con backpressure, circuit breaker con máquina de estados completa, reconexión con backoff exponencial, health checks de tres niveles, y apagado ordenado. El criterio R6 del ROADMAP ("*un monitor roto no tumba el resto*") se cumple mediante el patrón de circuit breaker con detección de gap/fault visible en Grafana.

En cuanto a la seguridad informática, el sistema implementa 15 defensas específicas contra ataques identificados en el modelo de amenazas, incluyendo: Argon2id para hashing de contraseñas, JWT con refresh tokens rotativos y detección de reuso, MFA TOTP con throttle, RBAC granular por recurso:acción, cifrado ChaCha20-Poly1305, zeroización de secretos en memoria, auditoría PHI inmutable con retención de 6 años, y endurecimiento contra inyección, XSS y DoS. El paquete de compliance (SPEC-034) mapea controles a HIPAA, ISO 27001 y NIST 800-53.

En cuanto a la validación clínica, los 152 tests del backend verificados el 17 de septiembre de 2026 (85 lib + 35 E2E + 32 HL7) pasan sin fallos, complementados por 66 property-based tests, 4 targets de fuzzing, y 13 vectores de referencia clínica (7 APACHE II con match exacto contra Knaus 1985 y 6 GCS contra Teasdale & Jennett 1974). La suite de conformidad detectó 3 errores aritméticos en vectores calculados a mano, demostrando la eficacia de la automatización para prevenir errores humanos.

En cuanto al rendimiento, el tiempo de arranque se mantiene por debajo de 140 ms (mínimo 111 ms), los endpoints de observabilidad alcanzan latencias submilisegundo (mediana 0,26–1,13 ms), y el endpoint más pesado (`/obs/metrics` con 33 familias de métricas) procesa más de 880 req/s con un p95 de 1,55 ms.

En cuanto a la metodología, Spec-Driven Development (SDD) demostró ser más adecuada que Scrum para el dominio médico, al proporcionar contratos formales verificables (criterios Gherkin), gates de calidad automatizados, y trazabilidad completa de requisito a implementación. Las 35 especificaciones (SPEC-001 a SPEC-035) documentan exhaustivamente cada decisión de diseño, su justificación clínica, y los criterios de aceptación medibles.

Limitaciones reconocidas: El binario actual (18,2 MiB) supera la meta publicada en el README de ~8 MiB debido a las dependencias acumuladas (HL7, FHIR, SurrealDB, ML); las pruebas de carga con usuarios concurrentes, el fuzzing prolongado y la cobertura instrumentada no se re-ejecutaron en esta sesión; NEWS2, SOFA y SAPS III aún no cuentan con vectores de referencia bibliográficos completos; el motor de similitud usa barrido por tenant (adecuado hasta ~10.000 pacientes); y el despliegue local opera sobre HTTP directo sin TLS automático, requiriendo proxy para exposición en red externa.

Recomendaciones para el piloto:

1. Ejecutar en staging los escenarios k6 y el fuzzing prolongado; registrar
 - métricas de disponibilidad a 30 días ligadas a los criterios R1–R8.
2. Completar los vectores clínicos con cita para NEWS2, SOFA y SAPS III.
3. Fijar TLS obligatorio en despliegues externos (proxy Caddy) y ejecutar
 - el drill de recuperación de forma trimestral.
4. Confirmar en commit las mejoras de interfaz actualmente en el árbol de
 - trabajo y añadir pruebas de UI para flujos de medición.
5. Re-evaluar el índice de similitud (HNSW) y el servicio ML (ONNX) al
 - superar ~10.000 pacientes.

Bibliografía

Angulo Peña, R. A. (2019). *Agente inteligente basado en Redes Neuronales Artificiales para la identificación de los determinantes de la estadía de pacientes en la Unidad de Cuidados Intensivos del HUAPA Cumaná, estado Sucre*. Trabajo Especial de Grado. Licenciatura en Informática, Universidad de Oriente, Núcleo de Sucre.

Chávez, V. (2012). Caracterización epidemiológica de la unidad de cuidados intensivos del hospital universitario de Coro "Dr. Alfredo Van Grieken". *Revista de la Universidad del Zulia*, 5(12), 129.

Klabnik, S. y Nichols, C. (2021). *The Rust Programming Language* (2.^a ed.). No Starch Press.

Knaus, W. A., Draper, E. A., Wagner, D. P. y Zimmerman, J. E. (1985). APACHE II: A severity of disease classification system. *Critical Care Medicine*, 13(10), 818–829. <https://doi.org/10.1097/00003246-198510000-00009>

Martínez, L. L., Yusmani, I. y De la Torre, A. (2020). Valoración del APACHE II inicial en la unidad de cuidados intensivo emergente. *Revista de Ciencias Médicas de Pinar del Río*, 24(3), e4418.

Schwaber, K. y Sutherland, J. (2017). *The Scrum Guide*. Scrum.Org. <https://scrumguides.org/>

Teasdale, G. y Jennett, B. (1974). Assessment of coma and impaired consciousness: A practical scale. *The Lancet*, 304(7872), 81–84. [https://doi.org/10.1016/S0140-6736\(74\)91639-0](https://doi.org/10.1016/S0140-6736(74)91639-0)

Velásquez, L. y Sánchez, M. (2002). Estancia prolongada en terapia intensiva: predicción y consecuencias. *Revista de la Asociación Mexicana de Medicina Crítica y Terapia Intensiva*, 16(2), 41–47.

Health Insurance Portability and Accountability Act (HIPAA). (1996). 45 CFR Part 164 — Security and Privacy. U.S. Department of Health and Human Services.

International Organization for Standardization. (2022). ISO/IEC 27001:2022. *Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. ISO.

National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations* (NIST SP 800-53 Rev. 5). U.S. Department of Commerce.

Open Web Application Security Project (OWASP). (2021). *OWASP Application Security Verification Standard (ASVS) v4.0*. <https://owasp.org/www-project-asvs/>

RFC 6238. (2011). *TOTP: Time-Based One-Time Password Algorithm*. Internet Engineering Task Force (IETF). <https://tools.ietf.org/html/rfc6238>

RFC 7519. (2015). *JSON Web Token (JWT)*. Internet Engineering Task Force (IETF). <https://tools.ietf.org/html/rfc7519>

HL7 International. (2020). *HL7 Version 2.4: Messaging Standard*. <https://www.hl7.org/implement/standards/productbrief.cfm?productid=51>

Health Level Seven International. (2021). *FHIR Release 4 (v4.0.1)*. <https://www.hl7.org/fhir/>

Ley 118 de 2021. Protección de datos personales. República de Cuba.

Decreto-Ley 370 de 2019. Informatización de la sociedad. República de Cuba.

Anexos

Anexo A: Estructura del Repositorio

Ruta	Contenido
<code>dmart-shared/</code>	Modelos, escalas clínicas, validación, ML (DecisionTree)
<code>dmart-server/</code>	Servidor Axum, 23 submódulos de API, HL7, FHIR, seguridad, observabilidad, migraciones, fuzz
<code>dmart-app/</code>	Frontend Leptos/WASM, componentes, páginas, PWA
<code>specs/</code>	35 especificaciones SDD (SPEC-001...035)
<code>docs/</code>	Arquitectura, API, escalas clínicas, compliance y runbooks (16 archivos)
<code>scripts/</code>	18 scripts de despliegue, DR, compliance y coste
<code>tests/</code>	E2E (Playwright) y carga (k6)
Raíz	Dockerfile, docker-compose.*, helm/, flux/, .argocd/, prometheus/, grafana/, systemd/

Anexo B: Scripts de Operación

Script	Función
<code>backup.sh</code>	Backup diario de SurrealKV
<code>dr_backup.sh</code>	Backup de disaster recovery (incremental)
<code>dr_restore.sh</code>	Restauración desde backup
<code>dr_verify.sh</code>	Verificación de integridad del backup
<code>dr_drill.sh</code>	Drill de restauración completo
<code>deploy_blue_green.sh</code>	Despliegue Blue/Green con verificación
<code>deploy_canary.sh</code>	Despliegue Canary (5 → 25 → 100 %)
<code>deploy_rollback.sh</code>	Rollback a versión previa
<code>deploy_status.sh</code>	Estado del despliegue actual
<code>compliance_generate.sh</code>	Generación de evidencia de compliance
<code>compliance_check.sh</code>	Validación de controles de compliance
<code>risk_assessment.sh</code>	Evaluación de riesgos trimestral
<code>cost_report.sh</code>	Reporte mensual de costes
<code>cost_forecast.sh</code>	Proyección de costes
<code>cost_idle_detect.sh</code>	Detección de recursos inactivos
<code>cost_rightsize.sh</code>	Right-sizing de recursos
<code>check-coverage-thresholds.sh</code>	Gate de cobertura por módulo
<code>optimize-wasm.sh</code>	Optimización del bundle WASM

Anexo C: Métricas Prometheus

Las 33 familias de métricas expuestas incluyen:

- **HTTP/Sistema:** `http_requests_total`, `http_request_duration_seconds`,
 - `http_requests_errors_total`, `uptime_seconds`, `process_cpu_seconds_total`,
 - `process_resident_memory_bytes`.
- **Base de datos:** `db_connections_active`, `surreal_connection_pool`,
 - `surreal_query_duration_seconds`.

- **Autenticación/Seguridad:** `auth_login_total`, `auth_refresh_total`,
 - `auth_failures_total{reason}`, `rbac_denials_total{permission}`.
- **Clínica:** `patients_total{status}`, `patients_created_total`,
 - `patients_deleted_total`, `measurements_total`, `scales_calculated_total{scale}`.
- **ML:** `ml_predictions_total{model}`, `ml_accuracy_gauge{model}`,
 - `ml_model_load_duration_seconds`.
- **Realtime/Interop:** `sse_connections_active`, `hl7_messages_processed_total{source}`,
 - `hl7_messages_errors_total{source}`.
- **Ingesta:** `ingest_gap_total`, `ingest_invalid_total`,
 - `ingest_fault_devices`, `ingest_throttled_total`.

Anexo D: Reglas de Alerta Prometheus

18 reglas de alerta configuradas en `prometheus/rules/`:

- **Críticas:** `DMartServerDown`, `DMartHighErrorRate` (>1 % 5xx en escrituras),
 - `DMartHighLatencyP95` (>2 s), `DMartDatabaseDisconnected`,
 - `DMartProcessMemoryHigh` (>4 GiB).
- **Seguridad:** `LoginBruteForce` (>30/min), `RefreshTokenFailureBurst`,
 - `LoginSuccessRateDrop`, `RBACDenialBurst`.
- **Clínica/ML:** `ClinicalVolumeAnomaly`, `MLModelDrift`, `MLMetricsStale`,
 - `ScalesEngineErrors`.
- **HL7/Ingesta:** `HL7IngestErrorRate`, `HL7IngestPipelineDown`,
 - `SensorGap`, `SensorFault`, `Throttling`.

Anexo E: Criterios de Cobertura por Módulo

Módulo	Umbral	Estado
hl7/parser.rs	90 %	95,0 % ✓
hl7/mlp.rs	90 %	100,0 % ✓
hl7/ingest.rs	90 %	94,7 % ✓
shared/scales.rs	85 %	89,1 % ✓
shared/validation.rs	85 %	100,0 % ✓
shared/ml.rs	80 %	95,9 % ✓
Global	≥60 %	65 % ✓

dMart UCI — Informe Técnico Final · Septiembre 2026 Estado: rama main @ 1d6356f + mejoras de interfaz en el árbol de trabajo. 152 tests backend · 35 SPECS completadas · Arranque <140 ms · 0 advisories de seguridad.